AED com LLM — roteiro docente e chaves dos quatro cenários

Material do professor — não distribuir antes das apresentações

2026.2

Ritual simples de cada encontro

1. Apresente a teoria no quadro. 2. Entregue o cenário e dê 10–15 minutos para o aluno responsável desenhar pergunta, dados, cálculo e resposta. Os outros três atuam como revisores. 3. Compare a proposta do estudante e a resposta sugerida pela IA com a chave abaixo. 4. O responsável implementa em casa uma solução pequena e apresenta o código no encontro seguinte, em até cinco minutos. A chamada não deve interromper a atividade; reserve o tempo de apresentação no início do próximo encontro.

Encontro	Responsável		Cenário	Código apresentado
15/10	Aluno _____	1:	Cantina: receita e gráfico	22/10
22/10	Aluno _____	2:	Loja: métrica e ambiguidade	29/10
29/10	Aluno _____	3:	Cafeteria: narrativa com evidência	05/11
05/11	Aluno _____	4:	Perguntar, esclarecer ou parar	19/11

12/11 é EXPOFACOL, sem aula regular; por isso o último código é apresentado em 19/11. Não há quinto cenário: o encontro de 19/11 é para a apresentação do código do Aluno 4, integração dos quatro incrementos e avaliação do fluxo agentic.

Esqueleto que vale para os quatro cenários

Pergunta → dado e grão → métrica → cálculo em Python/SQL → validação → gráfico/resumo → narrativa. A LLM traduz ou explica; o programa calcula e limita as ferramentas. A resposta deve incluir unidade, período e evidência. Se falta definição ou dado, perguntar ou parar.

Cenário 1 — Cantina: respostas e código de referência

Receita por produto: sanduíche $10 \times 8 + 12 \times 8 = 176$; suco $20 \times 4 + 15 \times 4 = 140$; fruta $8 \times 3 + 10 \times 3 = 54$. Maior receita observada: sanduíche, R$176 nas duas semanas. Isso não prova maior margem, preferência do cliente ou que preço/campanha causou o resultado.

Arquitetura no quadro: CSV (uma linha por produto e semana) → validar tipos/ausência → calcular receita por linha → agrupar por produto → barras ordenadas → duas frases sustentadas. Sem LLM, o script já automatiza a conta e o gráfico; uma LLM pode interpretar a pergunta e redigir, mas não muda a definição de receita.

```
import pandas as pd
df = pd.DataFrame([
    (1,"Sanduiche",10,8),(1,"Suco",20,4),(1,"Fruta",8,3),
    (2,"Sanduiche",12,8),(2,"Suco",15,4),(2,"Fruta",10,3)
], columns=["semana","produto","unidades","preco"])
df["receita"] = df.unidades * df.preco
totais = df.groupby("produto").receita.sum().sort_values()
print(totais)
totais.plot.bar(title="Receita␣observada␣por␣produto")
```

Aceitar `matplotlib`/`seaborn` em vez de `pandas.plot`. Conferir manualmente R$176, R$140 e R$54. Avaliação: grão e fórmula explícitos, resultado correto, gráfico rotulado, conclusão sem causalidade.

Cenário 2 — Loja: resposta exige métrica

Faturamento: A = R$1.000; B = R$1.200; C = R$1.050, então B lidera por faturamento. Valor médio por pedido: A 1000/50 = R$20; B 1200/80 = R$15; C 1050/35 = R$30, então C lidera por esse critério. "Melhor" não tem resposta única: pedir esclarecimento antes de selecionar.

Camada semântica mínima: `faturamento = soma do valor recebido na semana`; `valor_medio_pedido = faturamento / número de pedidos`; grão = semana; janela = A–C. Não confundir faturamento com lucro: custos e margem não existem nos dados.

```
dados = {"A": (1000,50), "B": (1200,80), "C": (1050,35)}
ticket = {semana: receita/pedidos
         for semana,(receita,pedidos) in dados.items()}
def responder(metrica):
    if metrica == "faturamento":
        return max(dados, key=lambda s: dados[s][0])
    if metrica == "valor_medio_pedido":
        return max(ticket, key=ticket.get)
    return "Você␣quer␣comparar␣faturamento␣ou␣valor␣médio␣por␣pedido?"
```

Um mapeamento por regra é suficiente para a demonstração; se houver LLM, ela pode devolver somente uma escolha dentre as duas métricas registradas. O código valida essa escolha. Testes: pergunta por faturamento, por valor médio e "qual é melhor?".

Cenário 3 — Cafeteria: narrativa baseada em evidência

Receita sobe de R$1.000 para R$1.200 (+20%); pedidos sobem de 50 para 80 (+60%); valor médio cai de R$20 para R$15 (−25%). Exemplo de narrativa válida: "Na semana 2, a receita foi 20% maior, enquanto o valor médio por pedido caiu 25%. O resumo não informa por que isso aconteceu." Não afirmar que uma campanha, produto ou comportamento causou a mudança.

Fluxo: Python calcula indicadores e IDs de evidência (E1 receita; E2 pedidos; E3 valor médio); serializa JSON; LLM redige até três frases; verificador confirma que IDs pertencem a E1–E3; interface mostra narrativa e números originais. Não enviar dados linha a linha quando os agregados bastam.

```
evidencias = {
  "E1": {"metrica":"receita", "s1":1000, "s2":1200,
         "variacao_pct":20},
  "E2": {"metrica":"pedidos", "s1":50, "s2":80,
         "variacao_pct":60},
  "E3": {"metrica":"valor_medio", "s1":20, "s2":15,
         "variacao_pct":-25}}
# Enviar evidencias + instrucao: usar apenas esses fatos;
# devolver claims com evidence_ids. Validar IDs antes de exibir.
import json, os, re
from openai import OpenAI
cliente = OpenAI() # le OPENAI_API_KEY do ambiente local
resposta = cliente.responses.create(
    model=os.getenv("OPENAI_MODEL", "gpt-5-mini"),
    instructions=("Escreva␣no␣maximo␣tres␣frases␣usando␣so␣o␣JSON.␣"
                  "Nao␣infira␣causas.␣Cite␣IDs␣E1,␣E2␣ou␣E3."),
    input=json.dumps(evidencias, ensure_ascii=False))
texto = resposta.output_text
ids_citados = set(re.findall(r"E\d+", texto))
if not ids_citados.issubset(evidencias):
    raise ValueError("A␣resposta␣citou␣evidencia␣inexistente")
print(texto)
```

O pacote oficial `openai` precisa estar instalado; a chave vem do ambiente local e o modelo pode ser escolhido pela variável `OPENAI_MODEL`. Se não houver conexão, a pessoa pode demonstrar uma resposta simulada, claramente identificada. Testes: números apoiados; pedido de causalidade; ID de evidência inexistente. Nunca publicar a chave no código ou no navegador.

Cenário 4 — perguntas: responder, esclarecer ou parar

Com os dados do cenário, A tem resposta: semana B, faturamento R$1.200. B pede esclarecimento entre faturamento e valor médio; se usar tabela do cenário 2, os líderes são diferentes. C não pode ser respondida causalmente: o conjunto não traz campanha, preço ou outras variáveis explicativas.
Ferramentas permitidas: `maior_faturamento()` e `calcular_valor_medio()`. O classificador pode escolher uma função conhecida ou retornar `clarificar/sem_evidencia`; nenhuma consulta SQL livre ou ação externa é necessária.

```
def decidir(pergunta):
    q = pergunta.lower()
    if "por␣que" in q or "causou" in q:
        return "Não␣há␣dados␣para␣identificar␣a␣causa."
    if "melhor" in q and "faturamento" not in q:
        return "Você␣quer␣faturamento␣ou␣valor␣médio␣por␣pedido?"
    if "faturamento" in q:
        return "Semana␣B:␣R$␣1.200␣(cálculo␣sobre␣a␣tabela)."
    return "Pergunta␣não␣mapeada;␣não␣executar␣ferramenta."
```

Este protótipo pode usar regras simples; a discussão agentic é o contrato de ferramenta, a decisão de parar e a validação do resultado, não uma cadeia complexa de agentes. Teste cada classe: suportada, ambígua e sem evidência.

Rubrica: 0,25 ponto para cada responsável

Pergunta, grão e métrica corretos: 0,05. Código simples executável: 0,10. Teste e evidência de saída: 0,05. Demonstração de até cinco minutos e resposta a uma pergunta de revisão: 0,05. Soma dos quatro responsáveis: 1,0 ponto. A avaliação não depende de framework, tela elaborada nem plano pago.

Referências para a conversa no quadro: Tukey (1977), *Exploratory Data Analysis*; Wickham & Grolemund (2017), *R for Data Science*; Lewis et al. (2020), Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks; documentação da Responses API e de ferramentas; documentação de Microsoft Fabric, Tableau Next, Amazon Quick, Looker e Databricks AI/BI. Conceitos avaliados: definição da pergunta, evidência, cálculo reproduzível, resposta sustentada e recusa quando os dados não respondem.